

SCR2043 OPERATING SYSTEMS

Name : WOO CHENG SHUAN
Student ID : A23CS0283
Section : 02

Marks

This lab assessment is designed to test your understanding and skills on some basic concepts and tools related to process monitoring and management in operating system. Please follow the instructions carefully and submit your answers in this word document and rename the file as **os-lab-assessment02-studentname-matricno.docx**.

Essential Steps Before Starting Lab Assessment 2:

1. Download necessary source codes:

Use the `wget` command to retrieve the following source code files to your Linux (or WSL or MacOS) environment:

```
wget -O mainprocess.c https://rebrand.ly/mainprocess_c
wget -O subprocess1.c https://rebrand.ly/subprocess1_c
wget -O subprocess2.c https://rebrand.ly/subprocess2_c
```

2. Compile the source files:

Use the `gcc` compiler to create executable files from the source code.

```
gcc mainprocess.c -o mainprocess
gcc subprocess1.c -o subprocess1
gcc subprocess2.c -o subprocess2
```

3. Execute the dummy processes:

Run all the dummy processes

```
./mainprocess &
```

Press **enter** two times.

4. The dummy processes are running for 2 hours. If you took longer than 2 hours on questions 1-9, please restart the main process with `./mainprocess &`.

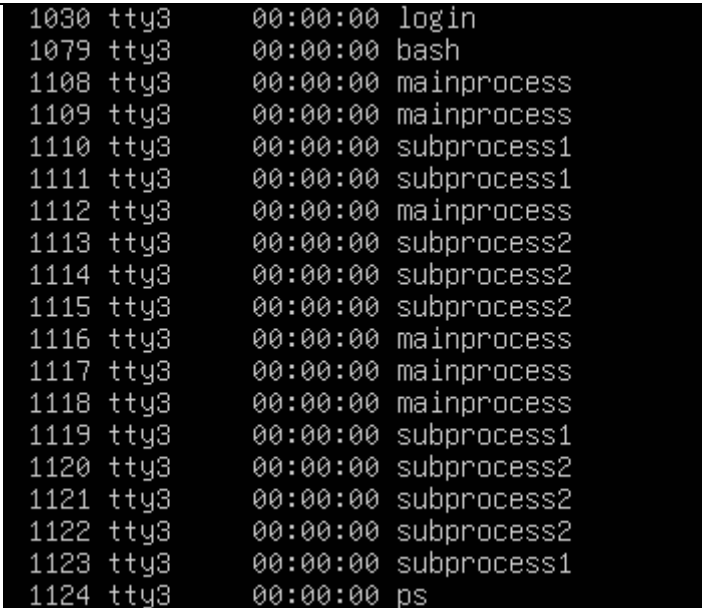
Lab Assessment 2 : Linux Process Monitoring and Management

Instructions:

1. Carefully execute each command as instructed in the questions.
2. Write down the exact command used for each task.
3. Capture a screenshot of the command's output.

Question 1

Use the `ps` command with the appropriate option to display a complete list of all running processes within the Linux operating system.

Command	
<code>ps -e</code>	
Output	
	1030 tty3 00:00:00 login
	1079 tty3 00:00:00 bash
	1108 tty3 00:00:00 mainprocess
	1109 tty3 00:00:00 mainprocess
	1110 tty3 00:00:00 subprocess1
	1111 tty3 00:00:00 subprocess1
	1112 tty3 00:00:00 mainprocess
	1113 tty3 00:00:00 subprocess2
	1114 tty3 00:00:00 subprocess2
	1115 tty3 00:00:00 subprocess2
	1116 tty3 00:00:00 mainprocess
	1117 tty3 00:00:00 mainprocess
	1118 tty3 00:00:00 mainprocess
	1119 tty3 00:00:00 subprocess1
	1120 tty3 00:00:00 subprocess2
	1121 tty3 00:00:00 subprocess2
	1122 tty3 00:00:00 subprocess2
	1123 tty3 00:00:00 subprocess1
	1124 tty3 00:00:00 ps

Question 2

Employ the `ps` command with necessary options to unveil comprehensive details about each running process.

Command
<code>ps -ef grep -E 'mainprocess subprocess'</code>
Output

```

woo@secr2043:~$ ps -ef|grep -E 'mainprocess|subprocess'
woo      1108      1079  0 16:00 tty3    00:00:00 ./mainprocess
woo      1109      1108  0 16:00 tty3    00:00:00 ./mainprocess
woo      1110      1109  0 16:00 tty3    00:00:00 ./subprocess1
woo      1111      1109  0 16:00 tty3    00:00:00 ./subprocess1
woo      1112      1108  0 16:00 tty3    00:00:00 ./mainprocess
woo      1113      1112  0 16:00 tty3    00:00:00 ./subprocess2
woo      1114      1112  0 16:00 tty3    00:00:00 ./subprocess2
woo      1115      1112  0 16:00 tty3    00:00:00 ./subprocess2
woo      1116      1079  0 16:01 tty3    00:00:00 ./mainprocess
woo      1117      1116  0 16:01 tty3    00:00:00 ./mainprocess
woo      1118      1116  0 16:01 tty3    00:00:00 ./mainprocess
woo      1119      1117  0 16:01 tty3    00:00:00 ./subprocess1
woo      1120      1118  0 16:01 tty3    00:00:00 ./subprocess2
woo      1121      1118  0 16:01 tty3    00:00:00 ./subprocess2
woo      1122      1118  0 16:01 tty3    00:00:00 ./subprocess2
woo      1123      1117  0 16:01 tty3    00:00:00 ./subprocess1
woo      1132      1079  0 16:04 tty3    00:00:00 grep --color=auto -E mainprocess|subprocess

```

Question 3

Use the `ps` command with some tools to only list processes named "subprocess" and show some info about them.

Command	
<code>ps -ef grep -E 'subprocess'</code>	
Output	
<pre> woo@secr2043:~\$ ps -ef grep -E 'subprocess' woo 1110 1109 0 16:00 tty3 00:00:00 ./subprocess1 woo 1111 1109 0 16:00 tty3 00:00:00 ./subprocess1 woo 1113 1112 0 16:00 tty3 00:00:00 ./subprocess2 woo 1114 1112 0 16:00 tty3 00:00:00 ./subprocess2 woo 1115 1112 0 16:00 tty3 00:00:00 ./subprocess2 woo 1119 1117 0 16:01 tty3 00:00:00 ./subprocess1 woo 1120 1118 0 16:01 tty3 00:00:00 ./subprocess2 woo 1121 1118 0 16:01 tty3 00:00:00 ./subprocess2 woo 1122 1118 0 16:01 tty3 00:00:00 ./subprocess2 woo 1123 1117 0 16:01 tty3 00:00:00 ./subprocess1 woo 1137 1079 0 16:05 tty3 00:00:00 grep --color=auto -E subprocess </pre>	

Question 4

Execute the `ps` command, specifying options that reveal only the following columns:

- Process ID (`pid`)
- Owner of the process (`user`)
- CPU percentage (`pcpu`)
- Memory percentage (`pmem`)
- Command (`cmd`)

Command	
<code>ps -eo pid,user,pcpu,pmem,cmd grep -E 'subprocess'</code>	
Output	

```

woo@secr2043:~$ ps -eo pid,user,pcpu,pmem,cmd|grep -E 'subprocess'
  1110 woo      0.0  0.1 ./subprocess1
  1111 woo      0.0  0.1 ./subprocess1
  1113 woo      0.0  0.1 ./subprocess2
  1114 woo      0.0  0.1 ./subprocess2
  1115 woo      0.0  0.1 ./subprocess2
  1119 woo      0.0  0.1 ./subprocess1
  1120 woo      0.0  0.1 ./subprocess2
  1121 woo      0.0  0.1 ./subprocess2
  1122 woo      0.0  0.1 ./subprocess2
  1123 woo      0.0  0.1 ./subprocess1
  1139 woo      0.0  0.2 grep --color=auto -E subprocess

```

Question 5

Building on the ps command used in Question 4, can you add an option to sort the listed processes by their memory usage (pmem)?

Command
<code>ps -eo pid,user,pcpu,pmem,cmd -sort=pmem grep -E 'subprocess'</code>
Output
<pre> woo@secr2043:~\$ ps -eo pid,user,pcpu,pmem,cmd --sort=pmem grep -E 'subprocess' 1110 woo 0.0 0.1 ./subprocess1 1111 woo 0.0 0.1 ./subprocess1 1113 woo 0.0 0.1 ./subprocess2 1114 woo 0.0 0.1 ./subprocess2 1115 woo 0.0 0.1 ./subprocess2 1119 woo 0.0 0.1 ./subprocess1 1120 woo 0.0 0.1 ./subprocess2 1121 woo 0.0 0.1 ./subprocess2 1122 woo 0.0 0.1 ./subprocess2 1123 woo 0.0 0.1 ./subprocess1 1146 woo 0.0 0.2 grep --color=auto -E subprocess </pre>

Question 6

Construct a command using ps, suitable options, and any additional tools to visualize the hierarchical structure (tree-like) of the following processes:

- "mainprocess"
- "subprocess1"
- "subprocess2"

Command
<code>Ps -forest -C mainprocess -C subprocess1 -C subprocess2</code>
Output

```
woo@secr2043:~$ ps --forest -C mainprocess -C subprocess1 -C subprocess2
  PID TTY          TIME CMD
 1116 tty3      00:00:00 mainprocess
 1117 tty3      00:00:00 \_ mainprocess
 1119 tty3      00:00:00 | \_ subprocess1
 1123 tty3      00:00:00 | \_ subprocess1
 1118 tty3      00:00:00 \_ mainprocess
 1120 tty3      00:00:00 \_ subprocess2
 1121 tty3      00:00:00 \_ subprocess2
 1122 tty3      00:00:00 \_ subprocess2
 1108 tty3      00:00:00 mainprocess
 1109 tty3      00:00:00 \_ mainprocess
 1110 tty3      00:00:00 | \_ subprocess1
 1111 tty3      00:00:00 | \_ subprocess1
 1112 tty3      00:00:00 \_ mainprocess
 1113 tty3      00:00:00 \_ subprocess2
 1114 tty3      00:00:00 \_ subprocess2
 1115 tty3      00:00:00 \_ subprocess2
```

Question 7

Use `pstree` command with option that show the number of threads to each process.

Command
<code>ps -c -s 2231</code>
Output
<pre>woo@secr2043:~\$ pstree -c -s 1116 systemd--login--bash--mainprocess--mainprocess--subprocess1 subprocess1 mainprocess--subprocess2 subprocess2 subprocess2</pre>

Question 8

Use `renice` command to change priority level of one of process “subprocess1”.

Command
<code>sudo renice -5 1110</code>
Output

```

woo@secr2043:~$ ps -o pid,nice,comm -C subprocess1
  PID  NI COMMAND
  1110   0 subprocess1
  1111   0 subprocess1
  1119   0 subprocess1
  1123   0 subprocess1
woo@secr2043:~$ sudo renice -5 1110
[sudo] password for woo:
1110 (process ID) old priority 0, new priority -5

```

Question 9

Terminate all running processes with the name “mainprocess”.

Command
killall -15 mainprocess
Output
<pre> woo@secr2043:~\$ killall -15 mainprocess Main process (ID: 1112) received signal: 15. Terminating... Main process (ID: 1108) received signal: 15. Terminating... Main process (ID: 1116) received signal: 15. Terminating... Main process (ID: 1118) received signal: 15. Terminating... Main process (ID: 1109) received signal: 15. Terminating... Main process (ID: 1117) received signal: 15. Terminating... [1]- Done ./mainprocess [2]+ Done ./mainprocess </pre>

Question 10

Write a short C or Python code (choose only one language) demonstrating multiprocessing with `fork()` and `wait()`. Compile and/or run the code. Show the output.

Source Code:

```

GNU nano 7.2                                                                    example.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

void child_process(){
    printf("Child process with PID: %d\n", getpid());
    int sleep_time = rand() %5 + 1;
    printf("Child process sleeping for %d seconds\n", sleep_time);
    sleep(sleep_time);
    printf("Child process existing\n");
}

int main(){
    printf("Parent process with PID: %d\n", getpid());

    //Fork a child process
    pid_t pid = fork();

    if (pid == 0){
        //this is the child process
        child_process();
        exit(0);
    }else if (pid > 0){
        //this is the parent process
        printf("Parent process waiting for child process to finish\n");
        //wait for the child process to finish
        wait(NULL);
        printf("Parent process existing\n");
    }else{
        //error occured while forking
        perror("fork");
        return 1;
    }

    return 0;
}

```

Output:

```

woo@secre2043:~$ gcc example.c -o example
woo@secre2043:~$ ./example
Parent process with PID: 1330
Parent process waiting for child process to finish
Child process with PID: 1331
Child process sleeping for 4 seconds
Child process existing
Parent process existing

```